SQL Cheat Sheet – ShopKart edition (Module 4)

Tool: sqliteonline.com → choose SQLite → File → Open DB → `shopkart.db`. Select one query, then Run (Shift+Enter).

The ShopKart schema

Table	Rows	Primary key	Columns
orders	5,000	order_id	order_date, customer_id (FK), product_id (FK), quantity, unit_price, discount, city, payment_mode, delivery_days, rating (NULL = not rated), returned ('Y'/'N')
customers	800	customer_id	name, gender, age, city, signup_date, membership ('Prime'/'Free')
products	60	product_id	product_name, category, list_price, cost_price

Revenue = `quantity * unit_price * (1 - discount)`. Profit = revenue – `quantity * cost_price`.

Query skeleton (the order you type)

```
SELECT   column, AGG(column) AS alias
FROM     table AS t
JOIN     other AS o ON t.key = o.key
WHERE    row_condition
GROUP BY column
HAVING   group_condition
ORDER BY alias DESC
LIMIT    10;
```

The order SQL runs it: FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. So: no COUNT/SUM in WHERE (use HAVING); no SELECT alias in WHERE (use a CTE).

SELECT basics

Need	Write
All columns, a few rows	`SELECT * FROM orders LIMIT 5;`
Some columns	`SELECT order_id, city FROM orders;`
Rename a column	`SELECT list_price AS price_rs FROM products;`

Need	Write
Calculated column	`SELECT quantity * unit_price * (1 - discount) AS revenue FROM orders;`
Unique values	`SELECT DISTINCT city FROM orders;`
Count unique	`SELECT COUNT(DISTINCT customer_id) FROM orders;`
Comment	`-- this line is ignored`

WHERE operators

Operator	Example
`=` `<>` `>` `>=` `<` `<=`	`quantity >= 4`
AND / OR (use brackets)	`returned = 'Y' AND (city = 'Pune' OR city = 'Kolkata')`
IN	`city IN ('Delhi', 'Mumbai', 'Bengaluru')`
BETWEEN (ends included)	`order_date BETWEEN '2025-11-01' AND '2025-11-30'`
LIKE (% any chars, _ one char)	`product_name LIKE '%phone%'`
IS NULL / IS NOT NULL	`rating IS NULL` (never `= NULL`)

Text in 'single quotes'. = on text is case sensitive; LIKE is not (English letters).

Sorting

`ORDER BY revenue DESC` (high → low), `ASC` is default. Multiple: `ORDER BY category, list_price DESC`. Top N: `ORDER BY ... DESC LIMIT N`.

Aggregates and GROUP BY

Function	Note
`COUNT(*)`	all rows
`COUNT(col)`	non-NULL values only
`SUM(col)`, `AVG(col)`, `MIN(col)`, `MAX(col)`	AVG ignores NULLs
`AVG(returned = 'Y')`	share of TRUE (TRUE = 1, FALSE = 0) → rate

```sql
SELECT city,
       COUNT(*) AS orders,
       ROUND(100.0 * AVG(returned = 'Y'), 1) AS return_rate_pct
FROM orders
GROUP BY city
HAVING COUNT(*) > 300
ORDER BY return_rate_pct DESC;
```

Rule: every SELECT column is either in GROUP BY or inside an aggregate.

JOINs

Join	Keeps
`INNER JOIN` (or `JOIN`)	only rows that match in both tables
`LEFT JOIN`	all rows of the left table; NULL where no match
`RIGHT JOIN`	all rows of the right table (= LEFT JOIN with tables swapped; SQLite 3.39+)
`FULL OUTER JOIN`	all rows of both tables (SQLite 3.39+)

```sql
-- 3-table join
SELECT o.order_id, c.name, c.membership, p.category
FROM orders AS o
JOIN customers AS c ON o.customer_id = c.customer_id
JOIN products  AS p ON o.product_id  = p.product_id;

-- Find what is missing: customers who never ordered (42)
SELECT c.customer_id, c.name
FROM customers AS c
LEFT JOIN orders AS o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

After a LEFT JOIN count with `COUNT(o.order_id)`, not `COUNT(*)`. Prefix columns that exist in both tables (`o.customer_id`).

CASE and functions

```sql
CASE
    WHEN quantity * unit_price * (1 - discount) < 1000 THEN 'Small'
    WHEN quantity * unit_price * (1 - discount) < 5000 THEN 'Medium'
    ELSE 'Large'
END AS order_size
```

Function	Example	Result
`ROUND(x, 2)`	`ROUND(1737.6000000000001, 2)`	1737.6
`UPPER(t)` / `LOWER(t)`	`UPPER('Smartwatch')`	SMARTWATCH
`LENGTH(t)`	`LENGTH('Smartwatch')`	10
`strftime('%m', d)`	`strftime('%m', '2025-11-14')`	'11'
`strftime('%Y', d)`	`strftime('%Y', '2025-11-14')`	'2025'
`COALESCE(a, b)`	`COALESCE(rating, 'Not rated')`	b when a is NULL

Do not use `COALESCE(rating, 0)` before AVG – missing is not zero (3.96 vs wrong 3.75).

Subqueries and CTEs

```sql
-- Subquery: products priced above the average (15 products)
SELECT product_name, list_price
FROM products
```

```sql
WHERE list_price > (SELECT AVG(list_price) FROM products);

-- CTE: named steps, read top to bottom
WITH per_customer AS (
    SELECT customer_id, COUNT(*) AS order_count
    FROM orders
    GROUP BY customer_id
)
SELECT AVG(order_count), MAX(order_count)
FROM per_customer;

-- Rank inside groups (window function, SQLite 3.25+)
RANK() OVER (PARTITION BY membership ORDER BY revenue DESC)
```

ShopKart numbers to check your work

Question	Answer
Orders / customers who ordered / products	5,000 / 758 / 60
Total revenue	₹1,47,04,649.70
Average order value	₹2,940.93
Top revenue city	Delhi ₹23,23,405.40
Pune return rate vs overall	32.4% vs 13.7%
Highest return-rate category	Fashion 21.0%
Customers who never ordered	42 (29 Free, 13 Prime)
Peak months	October ₹19,78,317, November ₹19,65,664 (festive season); lowest May ₹7,86,912
Top 3 categories, Prime and Free	Electronics, Fashion, Home

Common errors

Error	Fix
`no such column: Pune`	Put text in single quotes: `'Pune'`
`ambiguous column name`	Prefix with table alias: `o.customer_id`
`misuse of aggregate`	Move the COUNT/SUM condition from WHERE to HAVING
`near "FROM": syntax error`	Remove the comma before FROM
0 rows with `= NULL`	Use `IS NULL`
Millions of rows after JOIN	Check the ON condition